COMS W4771: Machine Learning

Lecture 23: Convolutional Networks

Tony Dear, Ph.D.

Department of Computer Science

School of Engineering and Applied Sciences

Topics

- Images and features
- Convolution and filters
- Convolutional networks
- Computer vision applications

Image Data

- Fully connected neural networks are not ideal for processing image data
- Very high-dimensional problem, assuming inputs are individual pixels

- Groups of *neighboring* pixels are related and may reveal useful features
- A fully connected network allows every pixel to interact with every other pixel, making it harder to discover useful representations

- We typically want functions $f(x)$ of images to be *equivariant* to geometric transformations $t(x)$ (e.g., translation or rotation): $f\big(t(x)\big) = t\big(f(x)\big)$
- No such guarantee in a general neural network

Image Features

- Instead of pixels, we may want more useful *representations* of the image
- **Features** in images are interest points that differ from the immediate neighborhood, e.g. in intensity, color, or texture

- Geometric features like edges and corners can provide semantic content
- Features are often the result of a specific *local group* of pixels

- We want a neural network that can detect and extract different features at different levels, followed by specific computer vision tasks

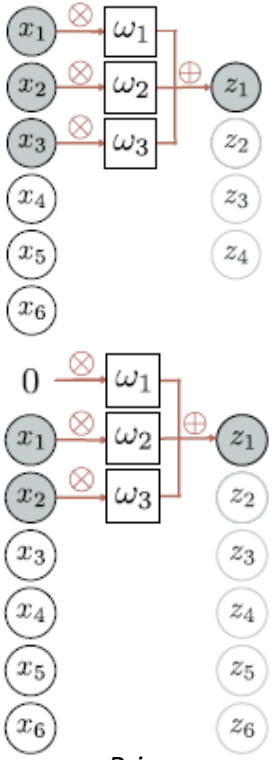
Convolution

- **Spatial filters** apply transformations to neighborhoods of pixels
- **Convolution** of a *kernel* $w \in \mathbb{R}^{2k+1}$ with an input $x \in \mathbb{R}^N$ is a "moving" dot product between w and subsequences of x:

$$z_i = (w * x)_i = w \cdot x_{i-k:i+k}$$

- The *kernel size* $2k + 1$ dictates the number of components used
- Larger captures a larger area, but requires more parameters

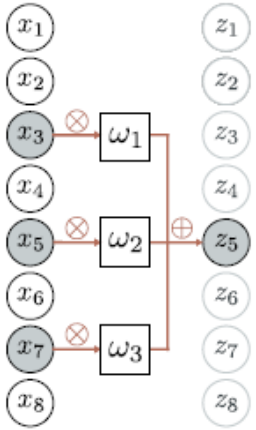
- For boundary components, can either use *zero padding*, or simply only keep outputs that fall within valid range



Prince

Convolution

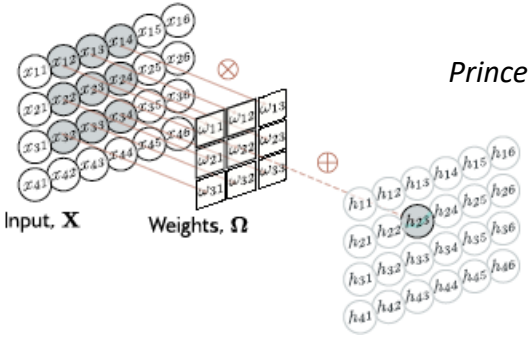
- Another option is *dilation*, or interspersing zeros within a larger kernel to reduce the number of parameters used



- A *stride* s greater than 1 allows the kernel to shift by more than one component at a time: $z_i = w \cdot x_{i-k:s:i+k}$

- For image data, an input sample X is a 2D grid of pixels
- So 2D convolution is a moving dot product between a *kernel matrix* $W \in \mathbb{R}^{(2k+1)\times(2l+1)}$ and X:

$$Z_{ij} = (W * X)_{ij} = W \odot X_{i-k:i+k,j-l:j+l}$$

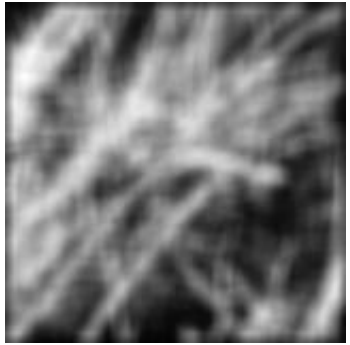


Prince

Examples of Filters

- *Moving average filter* $B = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

  blurs or smooths out an image
- Normalization maintains image brightness



- *Sharpening filter* $S = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

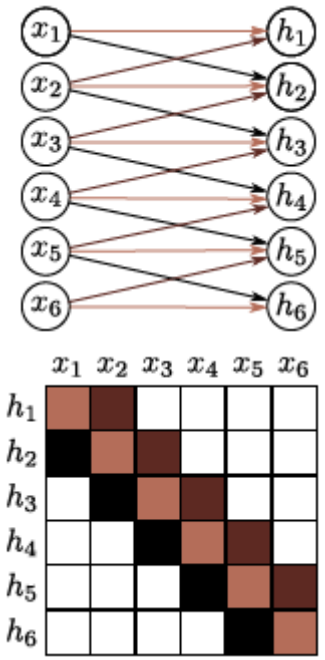
  accentuates edges

Convolutional Layers

- We can *learn* meaningful filters in neural networks
- A **convolutional layer** h takes in the convolution between a kernel and an input x (previous layer)

- For kernel size $2k + 1$ (and stride 1), each unit h_i only needs to be connected to the input units x_{i-k} to x_{i+k}
- Each h_i also shares the *same* set of kernel weights

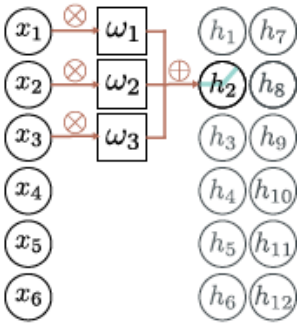
- Unlike a fully connected layer with $D_i D_h$ parameters, this layer only has $2k + 1$ parameters!



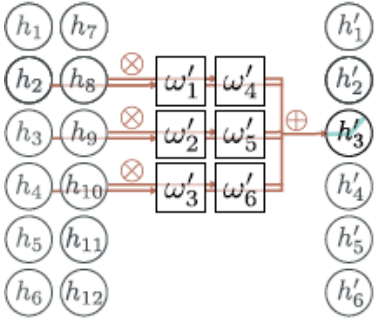
Prince

Channels

- We can apply multiple filters to the input by convolving with multiple sets of weights in parallel
- Each output is stored in a separate **channel** in the next layer



- To apply convolution to a multi-channel input, each *input* channel has its own kernel or set of weights
- Convolve each channel in parallel and then sum the results



- Multi-channel kernels can be represented in a matrix for 1D convolution or a *tensor* for 2D convolution

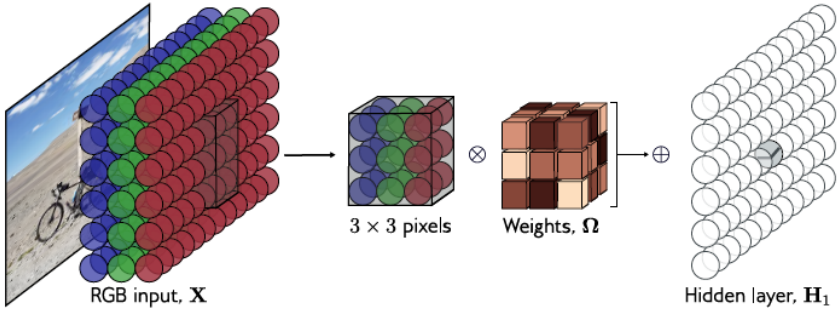
Prince

RGB Images

- Color images are naturally represented with 3 channels for red, green, blue
- Convolution is performed by sliding a 3D kernel tensor across the pixel grid, taking elementwise products, and summing the products

- Ex: If the kernel size is 3×3, we must apply a $3 \times 3 \times 3$ tensor to each 3×3 RGB image patch to output one hidden unit

- If we want more than one output channel, we can apply a 4D tensor (multiple 3D tensors in parallel)



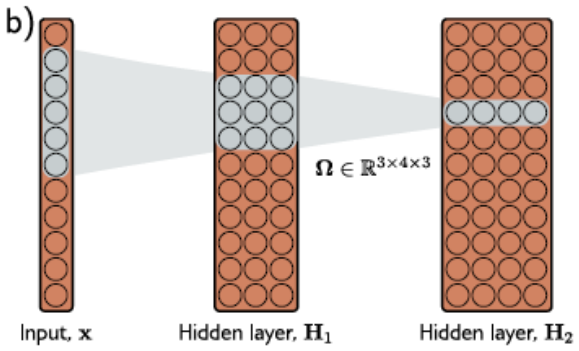
Prince

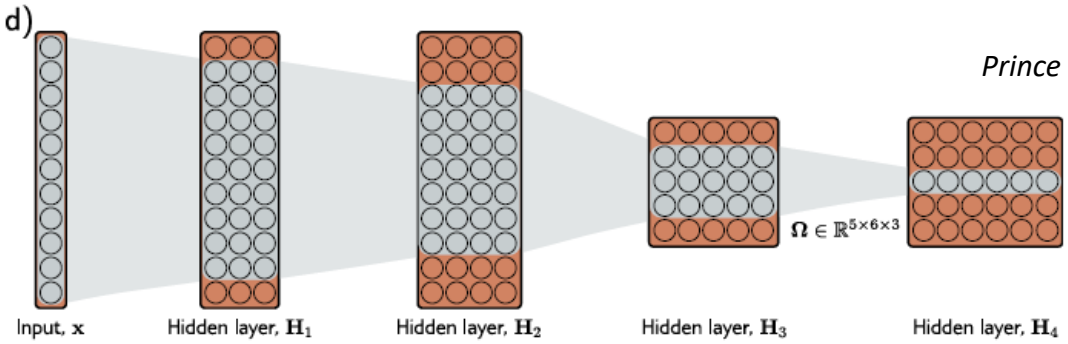
Receptive Field

- The **receptive field** is the input region that contributes to the input of a hidden unit

- Deeper layers have larger receptive fields, which allows them to learn complex features

- Early layers learn simpler (e.g., geometric) features
- Later layers learn more "semantic" features



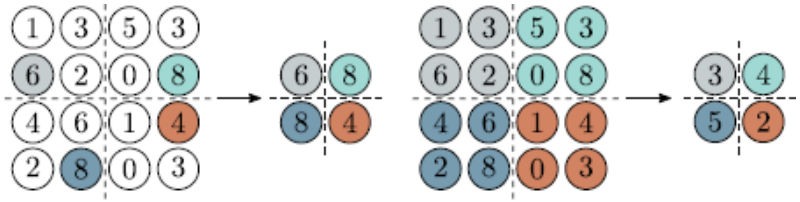


Pooling

- Convolution with stride greater than 1 reduces the next layer's dimensionality
- Generally, we can afford lower resolution deeper into the network, as we just need the features and not the original input

- Other *downsampling* methods include **max pooling** and **mean pooling**
- Every unit block of a certain size is replaced by their max or mean value

- Can also be implemented as a special layer in a convolutional network using weights and activation function

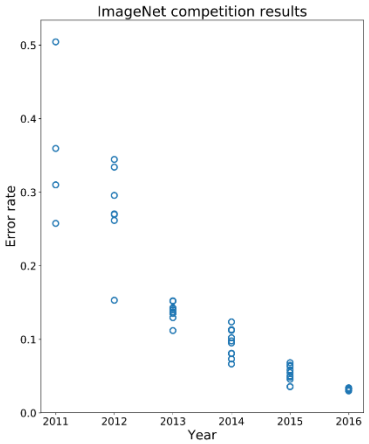


Prince

ImageNet

- ImageNet is a large repository of over 14M images, hand-annotated with over 20k categories (classes); some also contain attribute data or bounding boxes
- A “trimmed” subset of 1.2M images and 1k classes was used for the ImagetNet Large Scale Visual Recognition Challenge (ILSVRC), 2005-2017

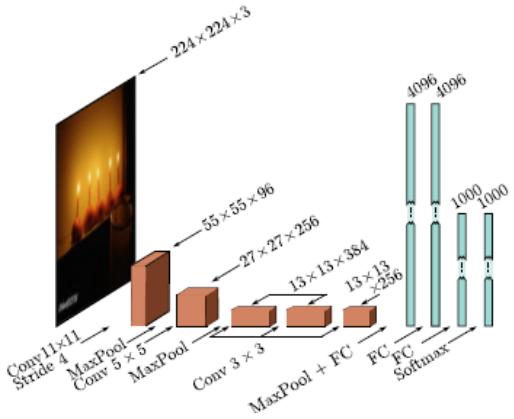
- Before 2012, best image classification results were achieved by SVMs with error rates as low as 25%
- In 2012, AlexNet was the first convolutional network entry
- Convnets continued to improve over the following years, eventually surpassing human performance



Wikipedia

AlexNet

- AlexNet consists of 5 convolutional layers, starting with a 11×11 kernel and shrinking to 3×3 kernels
- Channel sizes increase to 384, then shrink back to 256



Prince

- Each conv layer uses a ReLU activation, and some of them are followed by max pooling layers
- Finish with 3 fully connected and a softmax layer with 1000 units

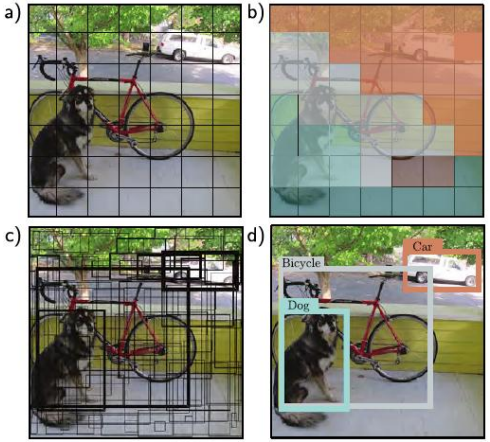
- Over 60M parameters, though most appear in the fully connected layers
- Trained on data set with 1.2M images by running momentum SGD

Object Detection

- *Object detection* is the task of identifying and localizing object(s) in an image
- You Only Look Once (YOLO) models solve this using convolutional networks

- First pass image through a convolutional network, and then shape the last layer into a $S \times S$ grid of cells by downsampling
- For each cell, predict a class (as in classification), but also output the positions and sizes of *bounding boxes*

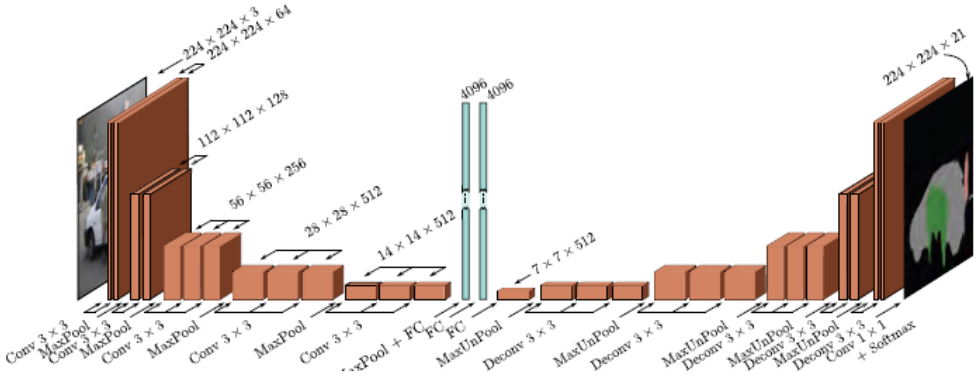
- For the output, typically only return the most confident bounding box found per object

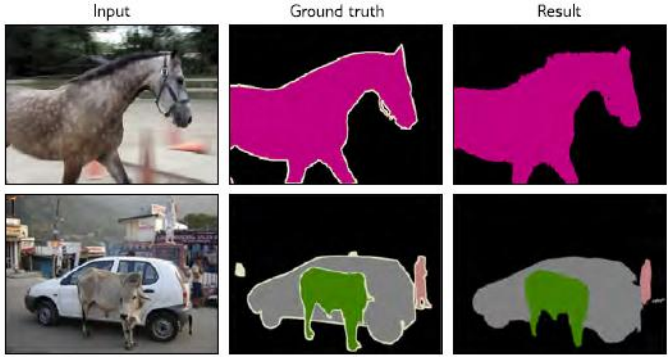


Prince

Semantic Segmentation

- *Semantic segmentation* is the task of assigning a label to each *pixel*
- An *encoder-decoder network* tries to understand *what* is in the image by downsampling, then tries to figure out *where* they are located by *upsampling*
- The output of this model is a tensor the same size as the input, along with a set of probabilities for each class in the 3rd dimension





Prince

Summary

- One important way to understand images is to extract features
- Hand-designed features are often insufficient for computer vision tasks
- Convolution with kernel filters can be used to perform feature mapping

- Convolutional networks exploit spatial locality and parameter sharing
- Deep networks expand receptive field and learn complex features
- We can also have multiple channels and apply pooling for further transforms

- CV tasks solved by convnets: Classification, object detection, segmentation